

```

class GopychMetamemoryCell(nn.Module):
    def __init__(self, dim=100,
n_patterns=1000):
        super().__init__()
        # Слой 1: Основная память (Hopfield
слой 1)
        self.patterns =
nn.Parameter(torch.randn(n_patterns, dim))
        # Слой 2: Метапамять (Hopfield слой
2)
        self.metapatterns =
nn.Parameter(torch.randn(n_patterns, dim))
        self.W1 = nn.Parameter(torch.randn(dim,
dim)) # Связи слоя 1
        self.W2 = nn.Parameter(torch.randn(dim,
dim)) # Связи слоя 2

    def forward(self, x):
        # Шаг 1: Импликативный поиск в
основной памяти
        h1 = torch.tanh(x @ self.W1) #
Активация слоя 1

```

```
dist1 = torch.cdist(h1.unsqueeze(0),  
self.patterns).min() # Лучший паттерн
```

```
# Шаг 2: Метапамять — активация  
слоя 2
```

```
h2 = torch.tanh(h1 @ self.W2) #  
Метапамять
```

```
dist2 = torch.cdist(h2.unsqueeze(0),  
self.metapatterns).min()
```

```
# Чувство знания = разница  
активаций
```

```
metamemory_strength = 1.0 - (dist1 +  
dist2) / 2
```

```
return h1, h2, metamemory_strength
```